

CH 2.2: SEMANTICS DATA CONTROL

Contents

Database Administrator (DBA)	1
Authentication	2
Access Rights (Authorization)	2
Semantic Integrity Constraints	3
View Management	4

Database Administrator (DBA)

DBA is a specialized IT professional responsible for **managing, maintaining, securing, and optimizing** databases in an organization. The DBA ensures that databases are **available, consistent, perform well**, and are **secure** from unauthorized access.

Key Roles and Responsibilities:

1. **Database Design:** Define schema structure, data types, relationships, and constraints.
2. **Installation & Configuration:** Install and set up DBMS software with proper connectivity.
3. **User Management:** Manage user roles and assign database access privileges.
4. **Backup and Recovery:** Schedule backups and plan for disaster recovery.
5. **Performance Tuning:** Optimize queries and indexes to boost performance.
6. **Security Management:** Protect data through access controls and encryption.
7. **Monitoring and Maintenance:** Track database health and apply regular updates.
8. **Data Integrity and Consistency:** Ensure data remains valid with rules and constraints.
9. **Capacity Planning:** Predict and plan for growing storage needs.
10. **Support and Troubleshooting:** Assist users and fix database-related issues.

Skills Required:

1. Proficiency in **SQL** and **DBMS tools**
2. Knowledge of **operating systems** and **networking**
3. Understanding of **data modeling** and **design principles**
4. Skills in **performance tuning, backup and recovery tools**
5. Awareness of **cybersecurity** principles

Common DBMS Tools a DBA Works With:

1. Oracle
2. Microsoft SQL Server
3. MySQL/MariaDB
4. PostgreSQL
5. MongoDB
6. IBM DB2

Authentication

Authentication is the process of verifying the identity of users who are trying to access the database.

Example: A student portal requires login credentials before allowing access to academic records.

- Ensures that only **authorized users** can log in.
- Common methods include **usernames & passwords**, **biometric scans**, or **two-factor authentication**.
- Helps prevent **unauthorized access** and protects sensitive data.

Authentication can be enforced in two levels -

Controlling Access to Client Computer - At this level, user access is restricted while login to the client computer that provides **user-interface** to the database server. The most common method is a **username/password** combination.

Controlling Access to the Database Software - At this level, the database administrator assigns some **credentials**(testimonials or certified documents showing that a person is entitled to credit)to the user. The user gains access to the database using these credentials. One of the methods is to create a **login account** within the database server.

Access Rights (Authorization)

Access rights define **what operations** a user is allowed to perform once authenticated. **Example:** In a hospital database, a doctor can view and update patient records, while a receptionist can only view them.

- Specifies **read, write, update, delete**, or **admin** rights.
- Can be assigned to **individual users or user groups**.
- Implemented through **roles, privileges, and permissions**.

Access Rights

✓ A user's access rights **refers to the privileges** that the user is given regarding DBMS operations such as the rights to create a table, drop a table, add/delete/update tuples in a table or query upon the table.

✓ In distributed environments, since there are **large number of tables and yet larger number of users**, it is not feasible to assign individual access rights to users. So, DDBMS defines **certain roles**. A role is a construct with certain privileges within a database system.

For example, the following SQL statements create a role "Accountant" and then assigns this role to user "ABC".

```
CREATE ROLE ACCOUNTANT;  
GRANT SELECT, INSERT, UPDATE ON EMP_SAL TO ACCOUNTANT;  
GRANT INSERT, UPDATE, DELETE ON TENDER TO ACCOUNTANT;  
GRANT INSERT, SELECT ON EXPENSE TO ACCOUNTANT;  
COMMIT;  
GRANT ACCOUNTANT TO ABC;  
COMMIT;
```

Semantic Integrity Constraints

Semantic Integrity Control ensures that the data stored in a database is **logically correct** and meaningful according to the rules and constraints of the application domain. It goes beyond simple structural correctness and focuses on the **validity of data values and relationships** based on business logic.

Types of Semantic Integrity Constraints

1. Data Type Integrity Constraint

- Ensures that each column in a table contains data of the correct **predefined type** (e.g., integer, varchar, date).
- Prevents insertion of invalid data types into columns.
- Example: A column defined to store age must only accept integers.

```
CREATE TABLE Employees (  
    emp_id INT,  
    name VARCHAR(100),  
    salary DECIMAL(10, 2) -- Ensures only numeric values with 2 decimal places  
);
```

This ensures `emp_id` is always an integer, `name` is a string, and `salary` is a decimal.

2. Entity Integrity Constraint

- Ensures that every table has a **primary key** and that this key is **unique and not null**.
- Maintains the **uniqueness** of each record in a table.
- Example: A student ID in a student table cannot be null or duplicated.

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Here, `student_id` is a **primary key** and must be **unique and not NULL**.

3. Referential Integrity Constraint

- Ensures that **foreign keys** in a table correctly reference **primary keys** in another table.
- Maintains consistency between related tables.
- Example: A foreign key in the 'enrollments' table referencing 'student_id' in the 'students' table must match a valid student ID.

```
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

CREATE TABLE Enrollments (
    student_id INT,
    course_id INT,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

This ensures that every `course_id` in `Enrollments` must exist in the `Courses` table.

View Management

View management is responsible for creating, storing, and maintaining **views** — virtual tables based on SQL queries. Views provide users with a **customized, simplified, or restricted** perspective of the database, enhancing **security, access control, and data abstraction**.

1. Views in Centralized DBMS

- **Definition:** A view is a virtual table generated by a query on base tables in the centralized database.
- **Purpose:** Helps restrict access to sensitive data by showing only selected columns/rows.
- **Example:**

```
CREATE VIEW EmployeeView AS
SELECT name, department FROM Employees WHERE role = 'Staff';
```

2. Updates Through Views

- **Update Possibility:** Simple views (based on a single table, without GROUP BY, JOIN, or aggregation) can usually be updated.
- **Restrictions:** Views involving multiple tables, joins, group functions, or DISTINCT are **not** usually updatable.
- **INSTEAD OF Triggers** (in some DBMSs): Used to handle updates on non-updatable views.
- **Example of updatable view:**

```
CREATE VIEW ActiveUsers AS
SELECT id, name FROM Users WHERE status = 'active';
```

3. Views in Distributed DBMS

- **Definition:** Views in distributed DBMSs may reference data from **multiple remote sites**.
- **Use:** Enable **location transparency** and **customized data access**.
- **Challenges:**
 - Update propagation across sites is complex.
 - View materialization and consistency become critical in a distributed setting.
- **Solution:** Use **global views** (defined over local schemas) and ensure **semantic correctness** through metadata synchronization and control mechanisms.